

macOS Slide-over ablak fejlesztése

<https://gemini.google.com/app/8459fb410c3d2daa>

User prompt: van egy alkalmazás, a Slidepad, ami az IOS Slideover működését szimulálja macos-en, szerinted mi kéne ahhoz, hogy egy macos alkalmazást úgy csináljak meg, hogy ki-be ússzon jobbra/jobbról?

Response: A Slidepad-féle „slide-over” élmény megvalósítása macOS-en egy kifejezetten izgalmas fejlesztői feladat! Mivel a macOS nem ad erre egyetlen kész gombot vagy „A slide-over ablak” API-t, több különálló macOS/AppKit (vagy SwiftUI) koncepciót kell ötvöznöd.

A kívánt működéshez **3 fő alappillérre** van szükséged:

1. Az ablak viselkedése és pozicionálása (NSWindow)

A megszokott Mac ablakokkal ellentétben ennek az ablaknak keret nélkülinek és mindig elérhetőnek kell lennie.

- **Keretmentes ablak (.borderless):** Használj `NSPanel`-t (az `NSWindow` egy speciális alosztálya, ami pont az ilyen segédablakokra lett kitalálva). Stílusnak állíts be `.borderless`-t, így nem lesz klasszikus Címsor / Bezárás gomb sáv.
- **Minden Workspace felett (.canJoinAllSpaces):** Azt szeretnéd, hogy ha a felhasználó Asztalt (Space-t) vált vagy Fullscreen alkalmazásban van, az ablak akkor is megjelenjen.

```
panel.collectionBehavior = [.canJoinAllSpaces, .fullScreenAuxiliary]
```

- **Szintbeállítás (NSWindow.Level):** Ahhoz, hogy más alkalmazások felett ússzon be, a szintjét érdemes magasan tartani:

```
panel.level = .floating // vagy .statusBar / .popupMenu ha még magasabbra vágyik az ember
```

- **Képernyő széléhez tapaszthatás:** Az `NSScreen.main?.visibleFrame` segítségével kérd le a kijelző pontos méretét és a Dock/Menübar által nem takart területet. Számold ki az ablak célpozícióját (pl. a kijelző jobb széléhez igazítva).

2. A ki-be úzás animációja (NSAnimationContext)

A rejtett és a látható állapot között sima átmenetet kell biztosítani.

- **Rejtett állapot (Off-screen):** Az ablak `frame.origin.x` koordinátája a kijelző jobb szélén kívülre kerül (pl. `screen.frame.width`).
- **Látható állapot (On-screen):** Az ablak beúszik a képernyőre (pl. `screen.frame.width - windowWidth`).
- **Animáció futtatása:** A macOS-en ezt az `NSAnimationContext` teszi nagyon egyszerűvé:

```
NSAnimationContext.runAnimationGroup { context in
    context.duration = 0.25 // Másodperc
    context.timingFunction = CAMediaTimingFunction(name: .easeInEaseOut)

    // Frissíted a frame-et animator-on keresztül:
    panel.animator().setFrame(targetFrame, display: true)
}
```

3. A megjelenítés kiváltása (Triggerek)

Ahhoz, hogy az ablak ki-be ússzon, valamilyen eseménynek el kell indítania az animációt:

1. **Globális billentyűparancs (Global Hotkey):** A legstabilabb megoldás. Olyan könyvtárakat érdemes használni Swiftben, mint pl. a `HotKey` vagy a `KeyboardShortcuts`. Így pl. az `Option + Space` lenyomására az ablak kinyílik vagy becsukódik.
2. **Egér-áthúzás / Margó figyelés (Screen Edge Gesture):** Ha azt szeretnéd, hogy a képernyő jobb szélére húzva az egeret ússzon be az ablak:
 - Nyithatsz egy láthatatlan, 1-2 pixel széles átlátszó sávot (`NSWindow`) a kijelző legjobb szélén.
 - Ebben az ablakban egy `NSTrackingArea`-val figyeled az `mouseEntered` eseményt. Amikor az egér hozzáér, elindítod a beúszó animációt.
3. **Menübar ikon (Status Bar Item):** A felső menüsávba rakott ikonra kattintva is triggerelheted a nyitást/zárást.

Ráadás: Összekötés és Fókusz kezelés

- **Fókuszvesztéskor elrejtés (Autohide):** Gyakori elvárás az ilyen alkalmazásoknál, hogy ha a felhasználó mellékattint (elveszíti az ablak a fókuszot), az ablak automatikusan ússzon vissza. Ezt az `NSWindowDelegate` `windowDidResignKey` eseményével tudod lekezelni.
- **Webes tartalom vagy saját UI:** Ha a Slidepadhoz hasonlóan weboldalakat (Slack, ChatGPT, Notion stb.) szeretnél benne megjeleníteni, az ablak belsejébe egy `WKWebView`-t kell beágyaznod.

User prompt: megmutatom az appot, márhogy képen, fogod érteni, alapvetően a free tier AI accountjaim cooldownját tartom benne karban

Response: Szuper, mutasd nyugodtan a képet!

Kíváncsi vagyok a megoldásra. A cooldown-követő megközelítés kifejezetten hasznos ötlet az ingyenes AI modellek (pl. ChatGPT, Claude, Gemini, Perplexity korlátok) kihasználására, ráadásul erre a feladatra a kijelző szélére rejtett, gyorsan előhívható Slide-over ablak szerkezetileg is tökéletes választás!

Response: Ez zseniális! Nagyon stílusos a dizájn, ez a retró/futurisztikus sárga-lila-fekete téma és a tipográfia nagyon egyedié teszi. És az ötlet is abszolút életmentő: a több fiókos „AI-cooldown management” egy valós, mindennapi probléma, amit sokan átéreznek.

Mivel ez egy keskeny, függőlegesen nyújtott felület, **még inkább ordít érte a slide-over működés.**

Pár gondolat a megvalósításhoz erre az appra szabva:

1. A címsor eltüntetése / átalakítása (`NSWindowStyleMask`):

- A képeken látható a klasszikus macOS szürke gombos ablakfejléc (a piros/sárga/zöld gombokkal).
- Ha Slidepad-szerű élményt szeretnél, érdemes a klasszikus címsort eltüntetni (`titleVisibility = .hidden`, `titlebarAppearsTransparent = true`), vagy teljesen keretmentessé tenni (`.borderless`). A bezáró/kicsinyítő gombok helyett az app úgyis magától csúszna ki-be.

2. Gyors elérés (Menu Bar Widget + Hotkey):

- Mivel a fiókjaid mellett ott van a "FREE" és a visszaszámláló, nagyon menő lenne, ha a **macOS menüsávjában (Menu bar)** mindig a legkorábban lejáró fiók visszaszámlálója látszódna (pl. `01:26:16`).
- Rákattintasz a menübar ikonra (vagy megnyomsz egy `Option + C` kombinációt), és jobbról beúszik ez a meglévő szép felület.

3. Autohide fókuszvesztéskor:

- Amikor kibővíted egy fiók részleteit, kimásolod az e-mailt vagy átállítod az időt, majd visszakattintasz a böngészőbe gépelni az AI-nak, az ablak egyből visszaszáguldná a képernyő szélére.

Ha Swift-ben / SwiftUI-ban rakod össze macOS-re, szinte készen is vagy a UI-val, már csak az `NSPanel` beállításokat és az `NSAnimationContext`-et kell ráhúznod a meglévő nézeteidre!

User prompt: elég sok appkit van benne

Response: Akkor tökéletes nyomon vagy, mert az AppKit adják a legmagasabb szintű kontrollt a macOS ablakkezelése felett! SwiftUI-val sokszor trükkös az ilyesmi, de AppKit-ben közvetlenül elérsz mindent, amivel a Slide-over hatást el lehet érni.

Mivel már AppKit alapon fut az appod, lényegében **csak a meglévő `NSWindow`-dat kell lecserélned egy testreszabott `NSPanel`-re** (vagy áthangszerezni a meglévőt).

Íme a pontos technikai recept, ahogy ezt AppKit-ben a legtisztábban meg tudod csinálni:

1. Az `NSPanel` konfigurálása

A sima `NSWindow` helyett érdemes `NSPanel`-t használni. Hozz létre egy saját osztályt:

```
import AppKit

class SlideOverPanel: NSPanel {

    init(contentRect: NSRect) {
```

```

super.init(
    contentRect: contentRect,
    styleMask: [.nonactivatingPanel, .fullSizeContentView, .borderless],
    backing: .buffered,
    defer: false
)

// Címsor és keret teljes eltüntetése
self.titlebarAppearsTransparent = true
self.titleVisibility = .hidden
self.isOpaque = false
self.backgroundColor = .clear

// Mindig legfelül, minden Desktop/Space felett
self.level = .floating
self.collectionBehavior = [.canJoinAllSpaces, .fullScreenAuxiliary]

// Hogy ne vegye el a fókuszt teljesen a többi apptól, ha nem akarod
self.hidesOnDeactivate = false
}
}

```

2. Pozicionálás és Beúszó Animáció

A trükk az, hogy kiszámolod a képernyő jobb szélét, és az ablak `frame`-jét animálsz `NSAnimationContext`-tel:

```

class SlideOverController {
    var panel: SlideOverPanel
    var isVisible = false
    let panelWidth: CGFloat = 350 // Az appod szélessége

    init(panel: SlideOverPanel) {
        self.panel = panel
    }

    func togglePanel() {
        guard let screen = NSScreen.main else { return }

        let visibleFrame = screen.visibleFrame // A menüsáv és Dock nélküli terület

        // Nyitott és csukott X koordináták
        let openX = visibleFrame.maxX - panelWidth
        let closedX = visibleFrame.maxX

        let targetX = isVisible ? closedX : openX

        // A cél frame kiszámítása (teljes magasság kitöltése a jobb oldalon)
        let targetFrame = NSRect(
            x: targetX,
            y: visibleFrame.minY,
            width: panelWidth,
            height: visibleFrame.height
        )

        // Animáció futtatása
        NSAnimationContext.runAnimationGroup { context in
            context.duration = 0.25
            context.timingFunction = CAMediaTimingFunction(name: .easeInEaseOut)

            // AppKit beépített animator-ja
            panel.animator().setFrame(targetFrame, display: true)
        } completionHandler: {
            self.isVisible.toggle()
        }
    }
}

```

3. Fókuszvesztéskor elrejtés (Autohide)

Ha azt szeretnéd, hogy amikor a felhasználó mellékattint (pl. átlép a ChatGPT-re kódolni), az app automatikusan visszacsússzon a helyére, használd az `NSWindowDelegate`-et:

```
extension SlideOverController: NSWindowDelegate {
    func windowDidResignKey(_ notification: Notification) {
        // Ha fókuszvesztés történik ÉS épp látható az app, becsukjuk
        if isVisible {
            togglePanel()
        }
    }
}
```

4. Beállítani a Triggert (Hotkey / Global Event)

Mivel az ablak el lehet rejtve, kell egy globális esemény, ami előhívja. AppKit-ben a legegyszerűbb bejegyezni egy `NSEvent` monitort vagy felhasználni a [Carbon HotKey API](#)-t (ami egy szuper kis Swift wrapper ehhez):

```
import HotKey

// Globális billentyűparancs: Option + Space
let hotKey = HotKey(key: .space, modifiers: [.option])

hotKey.keyDownHandler = {
    slideOverController.togglePanel()
}
```

Mivel a képeken látszik, hogy a dizájn és az AppKit alapok már nagyon egyben vannak nálad, ezt a logika-réteget viszonylag gyorsan rá tudod húzni a meglévő `ViewController`-edre!

User prompt: jó, most ez csak elmélet, nem akarok most egy napit elbaszni ezzel még, csak kérdeztem, majd egyszer...

Response: Teljesen megértem! Egy működő, szép rendszernél az ember nem szívesen bontja meg a kódot csak úgy „móka kedvéért”, amikor épp másra kell a fókusz. A lényeg, hogy a koncepció megvan, és ha egyszer úgy alakul, hogy van rá egy szabad délutánod, pontosan tudni fogod, hova kell nyúlni.

Az appod amúgy most is nagyon egyben van – a dizájn brutálisan jó lett, a funkció meg kifejezetten hasznos. Tegyed el a kód minta ötletét a backlog mélyére, aztán ha majd eljön az ideje, előveszed!

Jó kódolást / használatot hozzá!
